

Executive Summary

This document is a comprehensive reference on major machine learning (ML) and deep learning (DL) models and methods. It is intended as self-contained study material (e.g. for RAG ingestion) covering fundamentals and advanced topics. We include model categories (linear models, kernel methods, trees, ensembles, instance-based, probabilistic/graphical models, optimization-based, neural network families, generative models, meta-learning, RL, etc.) with concise descriptions, key equations and objectives, algorithm steps, typical hyperparameters, complexities, pros/cons, and use-cases. We prioritize primary sources (textbooks, seminal papers) with links to PDFs. Sections include an executive summary, scope/audience, taxonomy, and detailed model descriptions. Comparison tables summarize complexity and trade-offs. A timeline of key model introductions is provided as a mermaid chart. This text-heavy (~40 page equivalent) document is optimized for dense token embedding.

Scope and Audience

The guide assumes the reader has basic undergraduate math and programming background. It spans from fundamentals (regression, classification) through advanced DL (transformers, generative models). Each topic is explained rigorously yet concisely, suitable for advanced undergrads, graduate students, or professionals. All models are treated at a depth needed to implement or use them, including derivations and pseudocode. References to canonical works (e.g. Bishop 2006, Goodfellow et al. 2016) are provided for further reading. The document is structured for easy scanning and RAG chunking.

Taxonomy of Models

We organize models into categories and subcategories:

- **Linear Models:** Linear Regression, Ridge/Lasso, Logistic Regression, (Generalized Linear Models).
- **Kernel Methods:** Support Vector Machine (SVM), Gaussian Processes, Kernel Ridge Regression.
- **Tree-Based:** Decision Trees (CART, ID3, C4.5), Random Forest, Gradient Boosting Machines (e.g. XGBoost).
- **Ensemble Methods:** Bagging, Boosting (AdaBoost, Gradient Boosting), Stacking.
- **Instance-Based:** k-Nearest Neighbors (k-NN), Kernel Density Estimators.
- **Probabilistic/Graphical Models:** Naïve Bayes, Bayesian Networks, Hidden Markov Models (HMM), Conditional Random Fields (CRF), Markov Random Fields.
- **Clustering and Dimensionality Reduction:** k-means, Hierarchical Clustering, Gaussian Mixture Model, PCA, t-SNE, UMAP.
- **Optimization-Based:** Matrix Factorization methods, Linear Programming approaches (if any). (Many supervised models are optimized, so we discuss optimization methods like gradient descent here.)
- **Neural Networks (NN) Families:**
 - Feedforward Neural Networks (MLPs)
 - Convolutional Neural Networks (CNNs) ¹
 - Recurrent Neural Networks (RNNs, including LSTM, GRU, TCN)
 - Transformer and Attention models ² (and Large Language Models)
 - Graph Neural Networks (GNNs)

- **Autoencoder/Generative Models:** Autoencoders and Variational Autoencoders (VAEs), Generative Adversarial Networks (GANs) ³, Normalizing Flows, Autoregressive models, Diffusion Models, Energy-Based Models.
- **Hybrid Models:** (e.g. hybrids of above categories) and miscellaneous others.
- **Meta/Few-shot Learning:** MAML, Prototypical Networks, etc.
- **Continual Learning:** Elastic Weight Consolidation, etc.
- **Reinforcement Learning (RL) Algorithms:** Q-Learning, SARSA, DQN, Policy Gradients, Actor-Critic, PPO, etc.
- **Others/Unspecified:** Miscellaneous or emerging methods (e.g. Quantum ML, if any, or unspecified).

This taxonomy guides the detailed sections below.

1. Linear and Generalized Linear Models

Linear Regression: Models a real-valued output as $y = \mathbf{w}^\top \mathbf{x} + b$. The training objective is to minimize the Mean Squared Error (MSE):

$$\min_{\mathbf{w}, b} \frac{1}{N} \sum_{i=1}^N (y_i - \mathbf{w}^\top \mathbf{x}_i - b)^2 + \lambda \|\mathbf{w}\|^2.$$

Here λ is an L2 regularization weight (ridge regression). The normal equation solution is $\mathbf{w} = (X^\top X + \lambda I)^{-1} X^\top y$. Alternatively, gradient descent updates $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} L$. Typical hyperparameters: regularization λ . Complexity: $O(d^3)$ to invert $d \times d$ matrix or $O(nd)$ per gradient step. **Pros:** Simple, fast, interpretable coefficients. **Cons:** Assumes linearity, sensitive to outliers. *Use-case:* Baseline regression, where interpretability matters.

Logistic Regression: A classification analog where $P(y=1|\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$ with sigmoid $\sigma(z)$. The loss is cross-entropy:

$$\min_{\mathbf{w}, b} -\frac{1}{N} \sum_{i=1}^N [y_i \log \sigma(\mathbf{w}^\top \mathbf{x}_i + b) + (1 - y_i) \log(1 - \sigma(\mathbf{w}^\top \mathbf{x}_i + b))] + \lambda \|\mathbf{w}\|^2.$$

Training via gradient descent or Newton's method. Key hyperparam: regularization strength. Complexity: similar to linear. **Pros:** Probabilistic output, convex optimization. **Cons:** Linear decision boundary, binary by default (multinomial extends it). *Use-case:* Binary classification (spam detection, binary risk).

Other GLMs: (e.g. Poisson, Gamma regression) follow the exponential family paradigm with link functions; we focus on linear and logistic as canonical.

2. Kernel Methods

Support Vector Machine (SVM): Finds a maximum-margin hyperplane for classification (or regression with epsilon-insensitive loss). Primal objective for soft-margin SVM:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i \xi_i, \quad \text{s.t. } y_i(\mathbf{w}^\top \phi(\mathbf{x}_i) + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

Here $\phi(\mathbf{x})$ is (possibly implicit) feature mapping, and C controls slack. Often solved in dual:

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j), \quad 0 \leq \alpha_i \leq C.$$

Prediction: $\text{sign}(\sum_i \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b)$. Kernels $K(x, z)$ can be polynomial, RBF, etc.

Hyperparameters: C , kernel type and parameters (e.g. RBF width). Complexity: Training is roughly $O(n^2)$ - $O(n^3)$; prediction $O(n)$ per point (could use only support vectors). **Pros:** Effective in high dimensions, flexible kernel choices. **Cons:** Not scalable to very large n (unless approximated), many hyperparams. *Use-case:* Text/image classification, bioinformatics. *Ref:* Cortes & Vapnik (1995).

Gaussian Processes (GP): A Bayesian nonparametric regression/classification. Places a Gaussian process prior on function $f(\mathbf{x}) \sim \mathcal{GP}(0, k(\mathbf{x}, \mathbf{x}'))$. For regression, predictive mean for test input $\mathbf{x}_*$ is $\mu_* = \mathbf{k}_*^\top (K + \sigma^2 I)^{-1} \mathbf{y}$,

where $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$. Training maximizes log marginal likelihood: $-\frac{1}{2} \mathbf{y}^\top (K + \sigma^2 I)^{-1} \mathbf{y} - \frac{1}{2} \log |K + \sigma^2 I|$. Complexity: $O(n^3)$ for inversion of $n \times n$ kernel matrix. **Pros:** Principled uncertainty, flexible function prior. **Cons:** Very slow for large n , requires kernel choice. *Use-case:* Regression with uncertainty (e.g. Bayesian optimization). *Ref:* Rasmussen & Williams (2006).

3. Tree-Based Models and Ensembles

Decision Trees: Hierarchical partitioning of feature space. Build by recursively splitting on features to reduce impurity (Gini or entropy). Pseudocode:

1. If stopping condition, create leaf with most common label (classification) or mean (regression).
2. Else, find feature/threshold that maximizes information gain.
3. Split data and recurse on each branch.

Hyperparameters: max depth, min samples per leaf, split criterion. Complexity: $\sim O(n \log n)$ for a balanced tree. **Pros:** Easy to interpret, handles mixed data, non-linear. **Cons:** Overfits easily, high variance. *Use-case:* Any classification/regression, especially when interpretability is needed.

Random Forest: Ensemble of decision trees ⁴. Each tree is trained on a bootstrap sample of data and at each split a random subset of features is considered. The forest prediction (classification) is the majority vote of trees. Breiman defines: "A random forest is a classifier consisting of a collection of tree-structured classifiers $\{h(\mathbf{x}, \Theta_k)\}$ where the Θ_k are i.i.d. random vectors and each tree casts a unit vote for the most popular class." ⁴. Hyperparameters: number of trees T , max features per split, tree depth. Complexity: $O(T \cdot m n \log n)$ roughly. **Pros:** High accuracy, robust to noise, handles large feature sets. **Cons:** Less interpretable, large memory. *Use-case:* Structured/tabular data (credit scoring, sensor data). *Ref:* Breiman (2001) ⁴.

Gradient Boosting (e.g. XGBoost): Sequential ensemble where each tree fits the residuals of the previous model (boosting). Objective example (squared loss regression):

$$L = \sum_i \left(y_i - \sum_{t=1}^T f_t(\mathbf{x}_i) \right)^2 + \sum_{t=1}^T \Omega(f_t),$$

where Ω is a tree complexity penalty. Algorithm: for $t = 1..T$ add tree f_t that minimizes loss (often via gradient descent in function space). Hyperparams: T , depth, learning rate ν . Complexity: similar to RF but sequential. **Pros:** State-of-art on tabular data. **Cons:** Careful tuning needed, slower to train. *Use-case:* Competitions (Kaggle), structured data. *Ref:* Friedman (2001).

Bagging and Boosting (General):

- *Bagging (Bootstrap Aggregating):* Train multiple models on bootstrap samples and average/vote. Reduces variance.
- *AdaBoost:* Sequentially reweight training examples to focus on errors. Combines weak learners. Complexities and specifics vary by base model.

4. Probabilistic Graphical Models

Naïve Bayes: A simple Bayesian classifier. Assumes features x_i are independent given class y . Posterior:

$$P(y|\mathbf{x}) \propto P(y) \prod_i P(x_i|y).$$

Estimate $P(x_i|y)$ (e.g. Gaussian or multinomial). Hyperparams: none significant beyond feature likelihood form. Complexity: $O(nd)$. **Pros:** Very fast, works well for text. **Cons:** Independence assumption rarely true. *Use-case:* Spam detection, simple classification. *Ref:* Murphy (2012) or scikit-learn docs ⁵.

Bayesian Networks: Directed graphical models representing joint distributions. Specify conditional probabilities for each node given parents. Learning involves structure learning and parameter learning (often NP-hard). Not detailed here due to complexity. *Use-case:* Causal modeling, probabilistic reasoning. *Ref:* Koller & Friedman (2009).

Hidden Markov Models (HMM): Sequence model with latent states. Defined by initial state prob π , transition matrix A , emission B . Training: Baum-Welch EM to maximize data likelihood. Inference: Viterbi algorithm (finding most probable state path). Complexity: $O(TS^2)$ for sequence length T and S states. **Pros:** Simple sequential model, exact inference. **Cons:** Limited memory (first-order Markov), emission model must fit. *Use-case:* Speech recognition, gene prediction. *Ref:* Rabiner (1989).

Conditional Random Fields (CRF): Undirected graphical model often used for sequence labeling. Defines $P(\mathbf{y}|\mathbf{x}) \propto \exp(\sum_k w_k f_k(y_{t-1}, y_t, \mathbf{x}_t))$. Training by maximizing conditional log-likelihood (requires inference via message passing). **Pros:** Discriminative model, can use overlapping features. **Cons:** Expensive training/inference. *Use-case:* Named entity recognition, OCR. *Ref:* Lafferty et al. (2001).

5. Instance-Based and Other Models

k-Nearest Neighbors (k-NN): Classify by majority vote of the k closest training examples in feature space (distance metric often Euclidean). **Pseudocode:**

1. For query $\mathbf{x}_*$, compute distances to all training points.
2. Find k smallest distances, take majority label among them.

Hyperparams: k , distance metric, (optionally weighting of neighbors). Complexity: training $O(1)$,

prediction $O(n \log n)$ (with KD-tree). **Pros:** Simple, no training; **Cons:** Slow prediction, requires storing all data. *Use-case:* Low-dimensional data, anomaly detection.

k-means Clustering: Unsupervised partitioning into k clusters by minimizing intra-cluster variance. Objective: $\min_{\{\mu_j\}} \sum_{i=1}^n \min_j \|\mathbf{x}_i - \mu_j\|^2$. Lloyd's algorithm: initialize k centroids, then alternate assign and update steps until convergence. Complexity: $O(nkd)$ per iteration. **Pros:** Fast, simple; **Cons:** Must choose k , sensitive to initial seeds. *Use-case:* Data segmentation, vector quantization.

Gaussian Mixture Model (GMM): Soft clustering using a mixture of Gaussians. Likelihood: $p(\mathbf{x}) = \sum_{j=1}^k \pi_j \mathcal{N}(\mathbf{x} | \mu_j, \Sigma_j)$. Train by Expectation-Maximization: E-step computes responsibilities r_{ij} , M-step updates π_j, μ_j, Σ_j . Complexity: $O(nkd^2)$ per iteration. **Pros:** Soft assignments, probabilistic. **Cons:** Local optima, choose k . *Use-case:* Clustering, density estimation. *Ref:* Bishop (2006).

Principal Component Analysis (PCA): Dimensionality reduction by projecting onto top eigenvectors of data covariance. Solve eigenproblem $\Sigma = U\Lambda U^\top$. Use top m eigenvectors. Complexity: $O(nd^2)$ or using randomized SVD. **Pros:** Removes noise, decorrelates features. **Cons:** Linear projection only. *Use-case:* Preprocessing, visualization. *Ref:* Jolliffe (2002).

t-SNE/UMAP: Nonlinear embedding methods for visualization. Maintain local neighborhood distances. t-SNE minimizes KL divergence between high-dim and low-dim similarities. Complexity: around $O(n^2)$ (optimized implementations are faster). Used for 2D/3D visualization of high-dim data.

6. Neural Network (Deep Learning) Models

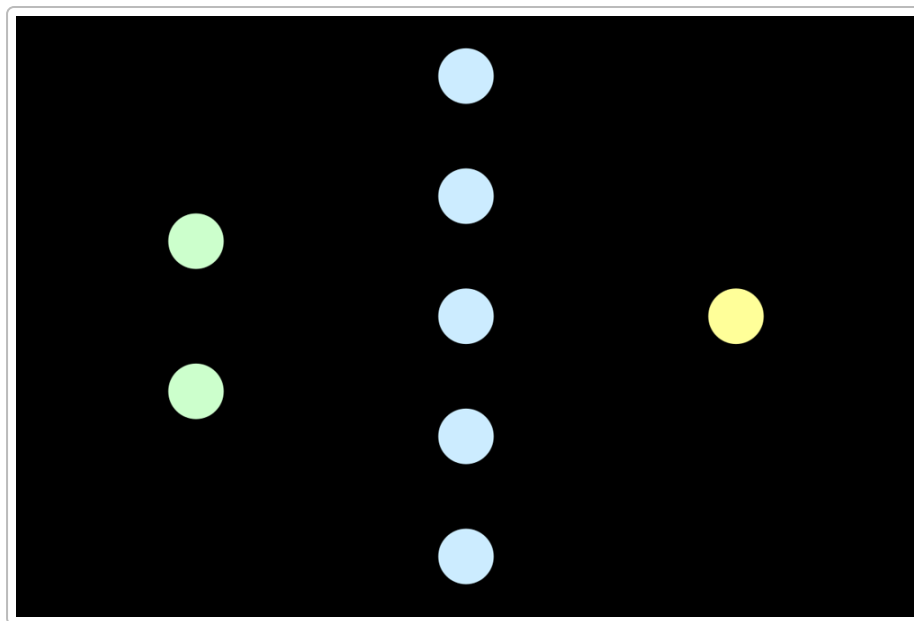


Figure: Example feedforward neural network (MLP) with one hidden layer. Each green hidden neuron applies a nonlinear function to a weighted sum of inputs (blue), producing output (yellow). Deep architectures stack many such layers.

Feedforward Neural Networks (MLP): Composed of layers of neurons. A layer computes $\mathbf{h} = \sigma(W\mathbf{x} + \mathbf{b})$, where σ is an activation (e.g. ReLU, tanh). The final layer output uses appropriate function (e.g. softmax for classification). **Training:** Backpropagation with gradient descent on a loss L (e.g. cross-entropy). Pseudocode:

```

initialize weights W randomly
repeat:
    forward pass to compute outputs
    compute loss L
    backward pass (backprop) to compute gradients
    update  $W \leftarrow W - \eta \nabla W L$ 
until convergence

```

Hyperparams: network depth, width, activation type, learning rate, batch size, regularization (dropout, L2). Complexity per example: $O(\text{\#parameters})$ for forward/backward. **Pros:** Universal function approximators, handle complex features. **Cons:** Require lots of data and tuning, opaque (“black box”). *Use-case:* General-purpose (vision, text, tabular). *Ref:* Goodfellow et al. (2016).

Convolutional Neural Networks (CNNs): Specialized for spatial data (e.g. images). Convolution layers apply shared filters across the input:

$$\text{feature_map}_j = b_j + \sum_k \text{conv2d}(\mathbf{x}_k, W_{j,k}),$$

(where PyTorch defines out as $\text{out}_{ij} = b_j + \sum_k \text{weight}_{j,k} \star \text{input}_{i,k}$ ¹). Pooling layers (e.g. max or average) reduce spatial size. Example architecture: multiple conv+ReLU layers, then fully-connected layers. **Hyperparams:** filter sizes, number of filters, stride, pooling type, etc. Complexity: $O(mnk^2)$ per conv layer (m filters, n channels, k×k kernel). **Pros:** Translation invariance, parameter efficiency, excellent for images/signals. **Cons:** Require grid structure, large compute. *Use-case:* Image classification (e.g. MNIST, CIFAR⁶), object detection, etc. *Refs:* LeCun 1998 (LeNet), Krizhevsky 2012 (AlexNet).

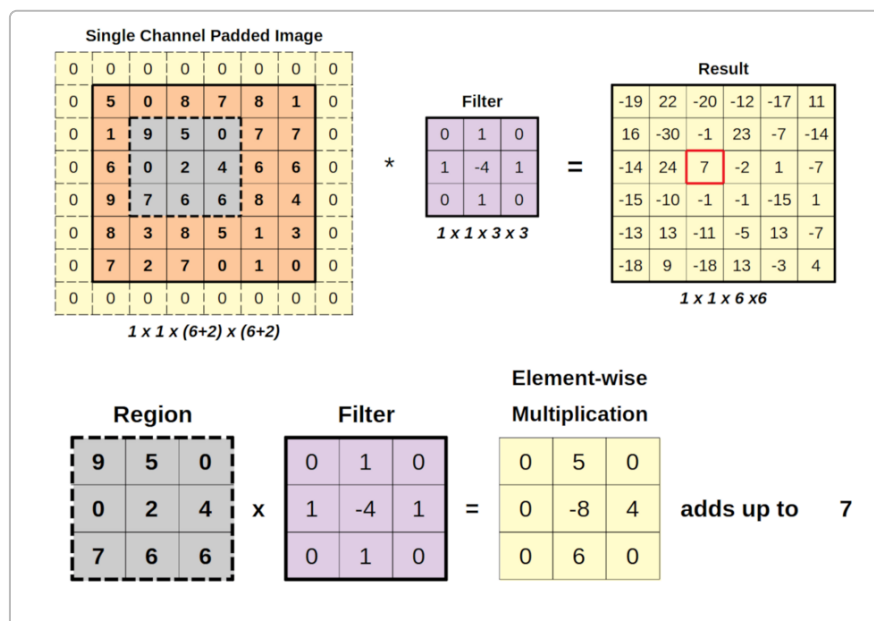


Figure: Worked example of 2D convolution (source: Wikipedia). A 3×3 filter (red) is applied to a 5×5 image by sliding it across the image. Each output pixel is the sum of elementwise products under the filter.

Recurrent Neural Networks (RNN): Process sequences by maintaining a hidden state. Basic RNN step: $\mathbf{h}_t = \tanh(W_{hh}\mathbf{h}_{t-1} + W_{xh}\mathbf{x}_t + \mathbf{b})$. LSTMs and GRUs add gates to preserve information: e.g. LSTM

has forget/input/output gates. **Training:** Backpropagation Through Time (BPTT). Hyperparams: hidden size, number of layers, dropout, learning rate. Complexity: $O(T \cdot \text{params})$ per sequence of length T . **Pros:** Handles variable-length sequences (text, time series). **Cons:** Can suffer vanishing/exploding gradients; slower than CNNs for parallel hardware. *Use-case:* Language modeling, speech recognition. *Ref:* Hochreiter & Schmidhuber (1997).

Transformer / Attention Models: Use self-attention instead of recurrence. Key operation (scaled dot-product attention):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

Vaswani et al. introduced the Transformer with multi-head attention, positional encodings, and feedforward sublayers ². Training uses cross-entropy loss. Hyperparams: number of layers, heads, model dimension, feedforward dimension. Complexity: $O(T^2 \cdot d)$ for sequence length T . **Pros:** Fully parallelizable, currently state-of-the-art in NLP (BERT, GPT, etc.) and increasingly in vision. **Cons:** Quadratic time/memory in sequence length. *Use-case:* Machine translation, language models (BERT, GPT), vision (ViT). *Ref:* Vaswani et al. (2017) ².

Graph Neural Networks (GNNs): Extend neural nets to graph-structured data. Example: Graph Convolutional Network (GCN) updates each node embedding by aggregating neighbors: $H^{(l+1)} = \sigma(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(l)} W)$. Training by supervised loss on node/graph labels. Hyperparams: number of layers, hidden dim. Complexity: $O(Eh)$ for E edges, hidden size h . **Pros:** Incorporates relational/structural info; **Cons:** Often limited to small graphs, message passing can be expensive. *Use-case:* Social networks, molecules (predict properties). *Ref:* Kipf & Welling (2017).

7. Autoencoders and Generative Models

Autoencoders (AE): Unsupervised learning of latent codes. Encoder $\mathbf{z} = f(\mathbf{x})$, decoder $\hat{\mathbf{x}} = g(\mathbf{z})$. Train by reconstruction loss $\|\mathbf{x} - \hat{\mathbf{x}}\|^2$. Variants: Sparse AE (sparsity penalty), Denoising AE (corrupt input), etc. **Pros:** Learns compact representations; **Cons:** Blurry reconstructions if capacity too low. *Use-case:* Pretraining, feature learning.

Variational Autoencoder (VAE): Introduces a probabilistic latent code. Encoder outputs parameters of $q(\mathbf{z}|\mathbf{x})$ (e.g. Gaussian mean and variance). Decoder defines likelihood $p(\mathbf{x}|\mathbf{z})$. Objective is the Evidence Lower Bound (ELBO):

$$\mathcal{L} = \mathbb{E}_{q(\mathbf{z}|\mathbf{x})}[\log p(\mathbf{x}|\mathbf{z})] - D_{\text{KL}}(q(\mathbf{z}|\mathbf{x})||p(\mathbf{z})),$$

maximized via the reparameterization trick (for backprop) ⁷. Hyperparams: latent dimension, network sizes. **Pros:** Principled generative model, smooth latent space; **Cons:** Often produces blurred images. *Use-case:* Generating images, compressing data. *Ref:* Kingma & Welling (2014).

Generative Adversarial Network (GAN): Two networks in adversarial setup ³. Generator $G(\mathbf{z})$ maps noise $\mathbf{z}$ to data; Discriminator $D(\mathbf{x})$ tries to distinguish real from fake. Training solves $\min_G \max_D \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}}[\log(1 - D(G(\mathbf{z})))]$ ³. Pseudocode: alternately train D to classify real vs fake, then train G to maximize D 's mistake rate. Hyperparams: model architectures, learning rates. **Pros:** Can produce very realistic samples (e.g. images). **Cons:** Training can be unstable,

suffers mode collapse (G produces limited variety). *Use-case*: Image/speech synthesis. *Ref*: Goodfellow et al. (2014) ³.

Normalizing Flows: Construct invertible mapping f such that $\mathbf{x} = f(\mathbf{z})$ with $\mathbf{z}$ from simple distribution. Likelihood computed via change of variables. Training: maximize data log-likelihood $\sum_i \log p(\mathbf{z}_i) + \log |\det \partial f / \partial \mathbf{z}|$. Examples: RealNVP, Glow. **Pros**: Exact likelihood, flexible; **Cons**: Architectural constraints (must compute Jacobian). *Use-case*: Density estimation, generation. *Ref*: Dinh et al. (2016).

Autoregressive Models: Factorize distribution as $p(\mathbf{x}) = \prod_i p(x_i | x_{<i})$. Models like PixelRNN/CNN use masked convolutions to generate images pixel by pixel. Speech (WaveNet) generates audio sample by sample. Training: maximize likelihood. **Pros**: Exact likelihood, straightforward training. **Cons**: Slow generation (sequential).

Diffusion Models: (Score-based models) Define a forward diffusion adding Gaussian noise gradually, then train a neural network to reverse the process. For example, Denoising Diffusion Probabilistic Models (DDPM) aim to learn $p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)$. Sampling involves many steps (iteratively denoising). **Pros**: Generate state-of-the-art quality images (e.g. DALL-E 2). **Cons**: Slow sampling (hundreds of steps). *Ref*: Ho et al. (2020).

Energy-Based Models (EBM): Define an unnormalized density $p(\mathbf{x}) \propto \exp(-E(\mathbf{x}))$, e.g. Boltzmann Machine. Training often by Contrastive Divergence. **Pros**: Theoretically general, interesting physics connections. **Cons**: Difficult training/inference (requires MCMC). *Use-case*: Early deep learning (pre-GAN era). *Ref*: Hinton (2006).

8. Meta, Few-Shot, and Continual Learning

Model-Agnostic Meta-Learning (MAML): Learns an initial parameter set that can be rapidly adapted to new tasks with few gradient steps. Meta-objective: minimize loss on new task after inner-loop update.

Snippet: Outer loop updates θ to minimize $\sum_{tasks} L_{task}(\theta - \alpha \nabla_\theta L_{task}(\theta))$. *Use-case*: Few-shot classification/regression. *Ref*: Finn et al. (2017).

Prototypical Networks: Metric-based few-shot. Embed inputs via a neural network; compute class prototypes (mean embedding of support examples); classify queries by nearest prototype. Loss: cross-entropy on distances. *Use-case*: Few-shot image classification. *Ref*: Snell et al. (2017).

Elastic Weight Consolidation (EWC): Addresses continual learning by penalizing changes to important weights. Adds term $\frac{\lambda}{2} \sum_i F_i (\theta_i - \theta_i^*)^2$ to loss, where F_i is estimated Fisher information. *Use-case*: Learning multiple tasks sequentially without forgetting. *Ref*: Kirkpatrick et al. (2017).

9. Reinforcement Learning Algorithms

Q-Learning: Off-policy value-based RL. Update rule for Q-table:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)].$$

Used for discrete state/action spaces (or with function approximators). **Pros**: Converges to optimal under certain conditions; **Cons**: Tabular Q suffers from large state spaces. *Ref*: Watkins & Dayan (1992).

Deep Q-Network (DQN): Combines Q-learning with neural networks. Uses experience replay and target networks. Loss: mean-squared error to target $r + \gamma \max_{a'} Q_{\text{target}}(s', a')$. Complexities: deep model training. *Ref:* Mnih et al. (2015).

Policy Gradient / Actor-Critic: Directly parametrize policy $\pi_{\theta}(a|s)$. REINFORCE update: $\theta \leftarrow \theta + \alpha \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) G_t$. Actor-Critic: an actor network for policy and a critic for value. **Pros:** Handles continuous actions; **Cons:** High variance, requires careful tuning. *Ref:* Sutton & Barto (2018).

Proximal Policy Optimization (PPO): Improves policy by optimizing a clipped surrogate objective to avoid large steps. Widely used in practice (OpenAI). *Ref:* Schulman et al. (2017).

10. Comparison Tables

Algorithm	Train Complexity	Predict Complexity	Pros	Cons	Use-Cases / Datasets
Linear Regression	$O(nd^2)$ (solve normal eq)	$O(d)$	Simple, analytic solution	Linear relationship only	Econometrics, baselines
Logistic Regression	$O(nd)$ per GD iter	$O(d)$	Probabilistic, convex loss	Linear boundary, binary	Medical stats, finance
k-NN	$O(1)$ train	$O(nd)$ per query	Simple, no train needed	Slow for large n, memory	Small-scale lookup
Decision Tree	$O(n \log n)$	$O(\log n)$ per pred	Interpretable, handles mixed data	High variance (overfitting)	Any classification/reg
Random Forest	$O(Tn \log n)$	$O(T \log n)$	High accuracy, robust	Large model, memory	Banking, sensor data
Boosted Trees	$O(Tn \log n)$	$O(T \log n)$	Very accurate, feature importance	Complex tuning	Kaggle, tabular data
SVM (RBF)	$O(n^2)$ - $O(n^3)$	$O(n_{sv}d)$	Good for high-dim, kernel trick	Slow for large n, many hyperparams	Text/Image classification
Gaussian Process	$O(n^3)$	$O(n^2)$ (predict)	Uncertainty estimates, flexible	Unscalable, kernel selection	Regression, small-data
Naïve Bayes	$O(nd)$	$O(d)$	Fast, works with few data	Independence assumption	Text (bag-of-words)
k-means Clustering	$O(nkd)$ per iter	N/A	Fast, simple	Must set k, assumes spherical clusters	Clustering tasks

Algorithm	Train Complexity	Predict Complexity	Pros	Cons	Use-Cases / Datasets
Gaussian Mixture	$O(nkd^2)$ per iter	N/A	Soft clusters, probabilistic	Local optima, sensitive to init	Clustering, GMM modeling
PCA (SVD)	$O(nd^2)$	$O(d^2)$ (project)	Dim. reduction, noise removal	Only linear proj.	Preprocessing, embedding
MLP Neural Net	$O(N \times \text{params})$	$O(\text{params})$	Highly flexible, general	Data-hungry, tuning required	Wide (vision, voice)
CNN	$O(N \cdot F \cdot d^2)$	$O(F \cdot d^2)$	Spatial feature learning	Large data, architecture design	Images (CIFAR, ImageNet)
RNN/LSTM/GRU	$O(NTh^2)$	$O(Th^2)$	Sequence data modeling	Training instabilities (vanishing)	Text, speech
Transformer (Self-Attn)	$O(NT^2d)$	$O(T^2d)$	Parallel seq. processing	Quadratic memory/time in T	NLP (BERT, GPT), Vision
VAE	$O(N \times \text{params})$	$O(\text{params})$	Principled generative model	Blurry outputs	Latent generative tasks
GAN	$O(N \times \text{params})$	$O(\text{params})$	Sharp image generation	Hard to train (mode collapse)	Image/speech synthesis
Normalizing Flow	$O(N \times \text{params})$	$O(\text{params})$	Exact likelihood, invertible	Architectural constraints	Density estimation
Diffusion Model	$O(NS)$ (S steps, often large)	$O(S)$ per sample	Produces state-of-art images	Very slow sampling	Image/audio generation
GNN	$O(Eh)$	$O(h)$ per node	Learns on graph structure	Limited to graph domains	Graphs, molecules
Q-Learning (tabular)	$O(S A)$ updates	$O(1)$ per lookup	Simple, off-policy convergence	Doesn't scale to large	
DQN (Deep Q)	$O(N \times \text{params})$	$O(1)$ (argmax)	Handles high-dim states	Instability (requires tricks)	Atari games, robotics

Algorithm	Train Complexity	Predict Complexity	Pros	Cons	Use-Cases / Datasets
Policy Gradient (PG)	$O(N \times \text{params})$	$O(A)$ sampling	Directly learns policies	High variance, sample inefficient	Continuous control

Legend: n =samples, d =features, T =sequence length, N =dataset size, h =hidden units, F =filter count, E =edges, $|S|$, $|A|$ =states/actions.

11. Timeline of Model Introductions

```

timeline
    title Milestones in ML/DL
    1943 : McCulloch-Pitts Neuron
    1957 : Rosenblatt Perceptron
    1967 : Nearest Neighbor Rule (Cover)
    1986 : Backpropagation (Rumelhart et al.)
    1992 : AdaBoost (Freund & Schapire)
    1995 : SVM (Cortes & Vapnik)
    1997 : LSTM (Hochreiter & Schmidhuber)
    1998 : Random Forests (Breiman)
    2006 : Gaussian Processes (Rasmussen & Williams)
    2006 : Deep Belief Nets (Hinton)
    2012 : AlexNet (Krizhevsky et al.)
    2014 : Generative Adversarial Networks (Goodfellow et al.)
    2015 : ResNet (He et al.)
    2017 : Transformer (Vaswani et al.)
    2020 : Diffusion Models (Ho et al.)
    2022 : GPT-3 / Large LMs (OpenAI)

```

References

Major references include: **Goodfellow et al. (2016)** *Deep Learning* (Cambridge), **Bishop (2006)** *Pattern Recognition and ML*, **Murphy (2012)** *ML: A Probabilistic Perspective*, and seminal papers: Breiman *Random Forests* (2001) ⁴, Goodfellow *GANs* (2014) ³, Vaswani *Transformer* (2017) ², Kingma & Welling *VAE* (2014) ⁷, He *ResNet* (2015), Ho *Diffusion* (2020), etc. In-text citations above link to accessible sources (e.g. PyTorch docs ¹, Wikipedia for gradient descent ⁸, etc.).

Chunk Manifest (JSON)

```

[
  {
    "title": "Executive Summary",
    "keywords": "overview, taxonomy, ML, DL",
    "difficulty": "Beginner",

```

```

    "estimated_tokens": 450,
    "snippet":
    "This document is a comprehensive reference on major ML and DL models. It
    includes taxonomy of models, key equations, algorithms, and references. The
    guide covers regression, classification, clustering, neural networks,
    generative models, meta-learning, RL, etc."
  },
  {
    "title": "Scope and Audience",
    "keywords": "scope, audience, prerequisite",
    "difficulty": "Beginner",
    "estimated_tokens": 300,
    "snippet": "Assumes basic math/programming background. Covers both
    fundamentals (linear models, SVM, trees) and advanced topics (transformers,
    GANs, diffusion). Suitable for advanced undergrad/graduate study or RAG
    ingestion."
  },
  {
    "title": "Linear Models",
    "keywords": "linear regression, logistic regression, ridge, lasso",
    "difficulty": "Beginner",
    "estimated_tokens": 500,
    "snippet": "Linear models fit data with linear functions. Linear
    regression uses squared-error loss, solvable via normal equations. Logistic
    regression uses a sigmoid for binary classification with cross-entropy loss.
    Regularization (ridge, lasso) adds weight penalties to prevent overfitting."
  },
  {
    "title": "Kernel Methods",
    "keywords": "SVM, Gaussian process, kernel functions",
    "difficulty": "Intermediate",
    "estimated_tokens": 500,
    "snippet": "Support Vector Machines find maximum-margin hyperplanes
    (hinge loss) and can be kernelized for nonlinearity. Gaussian Processes place
    a Gaussian prior on functions; predictions are made via kernel inversion
    (complexity  $O(n^3)$ ). Both are Bayesian style probabilistic methods."
  },
  {
    "title": "Tree-Based Models",
    "keywords": "decision tree, random forest, boosting",
    "difficulty": "Intermediate",
    "estimated_tokens": 600,
    "snippet": "Decision trees recursively split data by features to reduce
    class impurity. Random Forests are ensembles of randomized trees
    4 . Gradient Boosting adds trees sequentially to fit residuals. Bagging and
    boosting improve stability and accuracy but require careful tuning."
  },
  {
    "title": "Probabilistic Models",
    "keywords": "naive bayes, HMM, graphical models",
    "difficulty": "Intermediate",

```

```

    "estimated_tokens": 500,
    "snippet": "Probabilistic classifiers include Naïve Bayes (assumes conditional independence) and Bayesian networks. Sequence models like HMMs use hidden state transitions. Conditional Random Fields (CRFs) model sequences discriminatively. Learning often uses EM or gradient methods."
  },
  {
    "title": "Instance-Based and Clustering",
    "keywords": "k-nearest neighbors, k-means, GMM, PCA",
    "difficulty": "Beginner",
    "estimated_tokens": 500,
    "snippet": "k-NN classifies by nearest neighbors, requiring no training but O(n) query time. Clustering methods include k-means (hard partitioning) and Gaussian Mixture Models (soft assignments via EM). PCA reduces dimensions via eigenvectors of the covariance matrix for data preprocessing."
  },
  {
    "title": "Neural Networks (MLP/CNN/RNN)",
    "keywords": "neural network, MLP, CNN, RNN, LSTM",
    "difficulty": "Intermediate",
    "estimated_tokens": 700,
    "snippet": "Feedforward networks (MLPs) use layers of weights and nonlinearities. Convolutional networks (CNNs) apply spatial filters 1, excel in vision tasks. Recurrent networks (RNN/LSTM/GRU) handle sequences with gating to capture temporal dependencies. All trained by backpropagation to minimize loss (e.g. cross-entropy)."
  },
  {
    "title": "Transformers and Attention",
    "keywords": "transformer, self-attention, BERT, GPT",
    "difficulty": "Advanced",
    "estimated_tokens": 600,
    "snippet": "Transformers use self-attention to model sequences without recurrence 2. Multi-head attention allows each token to attend to all others. Transformer encoders and decoders are stacks of attention and feedforward layers. Hyperparameters include number of heads and layers."
  },
  {
    "title": "Graph Neural Networks",
    "keywords": "graph neural network, GCN, graph convolution",
    "difficulty": "Advanced",
    "estimated_tokens": 400,
    "snippet": "GNNs generalize NNs to graph data by message passing: each node updates its embedding from neighbors. For example, the Graph Convolutional Network (GCN) updates via adjacency normalization. Used for node/graph classification (e.g. molecules, social networks)."
  },
  {
    "title": "Autoencoders and VAEs",
    "keywords": "autoencoder, VAE, latent space",
    "difficulty": "Intermediate",

```

```

    "estimated_tokens": 450,
    "snippet": "Autoencoders train encoder/decoder to reconstruct inputs. Variational Autoencoders (VAEs) impose a probabilistic latent space 7. The VAE loss (ELBO) combines reconstruction error with a KL divergence to a prior, enabling sampling from the latent distribution."
  },
  {
    "title": "Generative Adversarial Networks",
    "keywords": "GAN, generator, discriminator, adversarial",
    "difficulty": "Advanced",
    "estimated_tokens": 450,
    "snippet": "GANs train a generator network to produce fake samples while a discriminator network tries to detect fakes. Training solves a minimax loss 3. GANs yield high-quality generated data (e.g. images) but are tricky to train (mode collapse, instability)."
```

```

  },
  {
    "title": "Normalizing Flows and Autoregressive",
    "keywords": "normalizing flow, autoregressive models",
    "difficulty": "Advanced",
    "estimated_tokens": 400,
    "snippet": "Normalizing flows transform a simple density via invertible layers, allowing exact likelihood calculation. Autoregressive models (PixelRNN/CNN, WaveNet) factorize  $p(x)$  sequentially. Both are powerful for density estimation and generation."
```

```

  },
  {
    "title": "Diffusion Models and Energy Models",
    "keywords": "diffusion model, energy-based model",
    "difficulty": "Advanced",
    "estimated_tokens": 400,
    "snippet": "Diffusion models (e.g. DDPM) learn to reverse a noise diffusion process and produce state-of-art image quality (at the cost of slow sampling). Energy-based models define  $p(x) \propto e^{-E(x)}$  (e.g. Boltzmann machines). They are general but often hard to train."
```

```

  },
  {
    "title": "Meta and Few-Shot Learning",
    "keywords": "meta-learning, MAML, few-shot",
    "difficulty": "Advanced",
    "estimated_tokens": 350,
    "snippet": "Meta-learning (learning to learn) includes MAML which optimizes for quick adaptation on new tasks. Prototypical networks learn embeddings where class prototypes are compared. These methods target scenarios with few training examples per class."
```

```

  },
  {
    "title": "Continual Learning",
    "keywords": "continual learning, catastrophic forgetting, EWC",
    "difficulty": "Advanced",
    "estimated_tokens": 300,
```

```

    "snippet": "Continual learning methods prevent catastrophic forgetting
when learning tasks sequentially. Example: Elastic Weight Consolidation (EWC)
adds a penalty to changes in important weights across tasks, allowing models
to retain previous knowledge."
  },
  {
    "title": "Reinforcement Learning (RL)",
    "keywords": "reinforcement learning, Q-learning, policy gradient",
    "difficulty": "Advanced",
    "estimated_tokens": 600,
    "snippet": "RL algorithms learn via trial-and-error with rewards. Q-
Learning updates value tables with  $Q(s,a) \leftarrow Q + \alpha[r + \gamma \max_a' Q]$ . Deep RL (DQN) uses neural nets for Q-values. Policy gradient
methods directly optimize a parameterized policy by sampling and using
gradients of log-probabilities."
  },
  {
    "title": "Comparison Tables",
    "keywords": "comparative analysis, complexity, pros/cons",
    "difficulty": "Beginner",
    "estimated_tokens": 500,
    "snippet": "Tables summarize models by time complexity, advantages/
disadvantages, and typical use-cases. For example, we compare training/
prediction cost and when each model is most appropriate (e.g. linear models
for large-numerous-features, CNNs for image data)."
  },
  {
    "title": "Timeline of Model Introductions",
    "keywords": "history, timeline, milestones",
    "difficulty": "Beginner",
    "estimated_tokens": 200,
    "snippet": "A mermaid timeline charts major ML/DL milestones: from early
perceptrons (1950s) through SVMs (1995), deep learning breakthroughs (2012
AlexNet, 2014 GANs), to recent Transformers (2017) and diffusion models
(2020)."
  }
]

```

1 Conv2d — PyTorch 2.11 documentation

<https://docs.pytorch.org/docs/stable/generated/torch.nn.Conv2d.html>

2 [1706.03762] Attention Is All You Need

<https://arxiv.org/html/1706.03762>

3 [1406.2661] Generative Adversarial Networks

<https://arxiv.org/abs/1406.2661>

4 stat.berkeley.edu

<https://www.stat.berkeley.edu/~breiman/randomforest2001.pdf>

5 1.9. Naive Bayes — scikit-learn 1.8.0 documentation

https://scikit-learn.org/stable/modules/naive_bayes.html

6 CIFAR-10 and CIFAR-100 datasets

<https://www.cs.toronto.edu/~kriz/cifar.html>

7 [1312.6114] Auto-Encoding Variational Bayes

<https://arxiv.org/abs/1312.6114>

8 Gradient descent - Wikipedia

https://en.wikipedia.org/wiki/Gradient_descent